Basic Django Interview Questions

1. What are Django URLs? URLs are one of the most important parts of a web application and Django provides an elegant way to design custom URLs with the help of its module known as URLconf (URL Configuration).

You can design your own URLs in Django and map them to Python view functions. These URLs can be static as well as dynamic. They are defined in `urls.py`, where they are matched with the equivalent view function.

Basic Syntax:

```
from django.urls import path
from . import views

urlpatterns = [
    path('data/2020/', views.data_2020),
    path('data/<int:year>/', views.data_year),
]
```

2. Explain the Django project directory structure.

- `manage.py` – Command-line utility that allows you to interact with your Django project. - `__init__.py` – An empty file that tells Python that the current directory should be considered a Python package. - `settings.py` – Contains configurations of the current project like database connections, installed apps, middleware, etc. - `urls.py` – All the URLs of the project are present here. - `wsgi.py` – Entry point for your application used by web servers to serve the project you created.

3. What are models in Django? A model in Django is a class that maps to a database table (or collection). Each attribute of a Django model class represents a database field. Models are defined in `app/models.py`.

Example:

```
from django.db import models

class SampleModel(models.Model):
    field1 = models.CharField(max_length=50)
    field2 = models.IntegerField()

    class Meta:
        db_table = "sample_model"
```

Every model inherits from `django.db.models.Model`. The `Meta` class lets you configure things like permissions, singular and plural names, table name, and whether the model is abstract.

4. What are templates in Django or Django template language? Templates are an integral part of the Django MVT architecture. They generally consist of HTML, CSS, and JavaScript, in which dynamic variables and information are embedded with the help of views.

A template is rendered with a *context*. Rendering replaces variables with their values (from the context) and processes tags. Everything else remains unchanged.

The Django template language has four main constructs:

- Variables - Tags - Filters - Comments

5. What are views in Django? A view function, or “view” for short, is a Python function that takes a web request and returns a web response. The response can be:

- HTML content of a web page - A redirect - A 404 error - An XML document - An image, etc.

Example:

```
from django.http import HttpResponse

def sample_function(request):
    return HttpResponse("Welcome to Django")
```

There are two types of views:

- **Function-Based Views (FBV)** – Views defined as functions. - **Class-Based Views (CBV)** – Views defined as classes, providing an object-oriented approach.

6. What is Django ORM? Django's ORM (Object Relational Mapper) lets you interact with the database in a Pythonic way, avoiding raw SQL queries. You can retrieve, save, delete, and perform other operations on the database using Python methods. It acts as an abstraction layer between models and the database.

7. Define static files and explain their uses. Websites often need to serve additional files such as images, JavaScript, or CSS. In Django, these are called **static files**. Django provides `django.contrib.staticfiles` to help manage these static assets.

8. What is Django REST Framework (DRF)? Django REST Framework is an open-source framework built on top of Django that allows you to create RESTful APIs quickly and efficiently.

9. What is `django-admin` and `manage.py`? Explain their commands.

- `django-admin` is Django's command-line utility for administrative tasks. - `manage.py` is a thin wrapper around `django-admin` automatically created in each Django project. It does the same tasks as `django-admin` but also sets the `DJANGO_SETTINGS_MODULE` environment variable to point to the project's `settings.py`.

Some important commands:

- `django-admin help` – Display usage info and list of available commands. - `django-admin version` – Check your Django version. - `django-admin check` – Inspect the project for common problems. - `django-admin compilemessages` – Compile `.po` files to `.mo` for translation. - `django-admin createcachetable` – Create cache tables for the database cache backend. - `django-admin dbshell` – Open the DB command-line client. - `django-admin diffsettings` – Show differences between your settings file and Django's default settings. - `django-admin dumpdata` – Dump data from the database. - `django-admin flush` – Flush all data from the database. - `django-admin inspectdb` – Generate Django models from existing database tables. - `django-admin loaddata` – Load data into the database from fixture files. - `django-admin makemessages` – Generate message files for translations. - `django-admin makemigrations` – Create new migrations based on model changes. - `django-admin migrate` – Apply migrations to synchronize the database with models. - `django-admin runserver` – Start the development web server. - `django-admin sendtestemail` – Send a test email to verify email settings. - `django-admin shell` – Start the Python interactive interpreter with Django loaded. - `django-admin showmigrations` – Show all migrations in the project. - `django-admin sqlflush` – Print SQL statements that would be executed by `flush`. - `django-admin sqlmigrate` – Print SQL for a specific migration. - `django-admin sqlsequencereset` – Print SQL to reset sequences for given apps. - `django-admin squashmigrations` – Squash a range of migrations. - `django-admin startapp` – Create a new Django app. - `django-admin startproject` – Create a new Django project. - `django-admin test` – Run tests for all installed apps. - `django-admin testserver` – Run development server using data from fixtures. - `django-admin changepassword` – Change a user's password. - `django-admin createsuperuser` – Create a superuser account. - `django-admin remove_stale_contenttypes` – Remove stale content types from deleted models. - `django-admin clearsessions` – Clean out expired sessions.

10. What is Jinja templating? Jinja (commonly Jinja2) is a popular templating engine for Python.

Key features:

- **Sandbox Execution** – Provides a protected environment, useful for safe template execution. - **HTML Escaping** – Automatically escapes characters like `<`, `>`, `&` to prevent XSS attacks. - **Template Inheritance** – Allows templates to extend and reuse base templates. - **Performance** – Generates HTML templates faster than Django's default engine in many cases. - **Debugging** – Easier to debug compared to the default Django template engine.

11. Explain Django architecture. Django follows the **MVT (Model–View–Template)** pattern, which is similar to MVC but with different naming:

- **Model** – Data access layer, defines the data structure. - **View** – Business logic; processes requests and returns responses. - **Template** – Presentation layer (HTML + DTL).

The "controller" aspect is largely handled by Django itself (URL routing and view dispatch). The developer provides the model, view, and template, maps them to URLs, and Django serves them to the user.

12. What is the difference between a project and an app in Django? - **Project** – The entire Django application configuration; it may contain multiple apps. - **App** – A module inside a project that handles one specific use case or functionality. - Example: A payment system app within an e-commerce project.

13. What are different model inheritance styles in Django?

- **Abstract Base Class Inheritance** – Parent class only holds common information that you don't want to repeat in each child model; no table is created for the abstract base. - **Multi-Table Model Inheritance** – Each model gets its own database table; used when subclassing an existing model and you need a separate table. - **Proxy Model Inheritance** – No additional table; used when you want to change Python-level behavior (methods, ordering) without changing the model's fields.

Intermediate Django Interview Questions

1. What's the use of Middleware in Django? Middleware runs between the request and response. It acts as a hook to process requests before they reach the view and responses before they reach the client. In Django, when a request is made, it passes through middleware to views and the response passes back through middleware to the client.

2. What is context in Django? Context is a dictionary mapping template variable names to Python objects. It is passed to a template so that variables in the template can be replaced with actual values.

3. What is `django.shortcuts.render` function? `render()` combines a given template with a context dictionary and returns an `HttpResponse` object.

Syntax:

```
from django.shortcuts import render

def my_view(request):
    context = {"key": "value"}
    return render(request, "template.html", context)
```

- `request` – HttpRequest object. - `template_name` – Name/path of the template. - `context` – Dictionary of data to pass to the template. - `content_type`, `status`, `using` – Optional parameters to control response type, HTTP status code, and template engine.

4. What's the significance of the `settings.py` file? `settings.py` stores project configurations such as:

- Database configuration - Installed applications - Middlewares - Template engines - Static and media file settings - Security keys - Allowed hosts, etc.

5. How to view all items in a Model?

```
ModelName.objects.all()
```

6. How to filter items in a Model?

```
ModelName.objects.filter(field_name="term")
```

7. What's the use of the session framework? The session framework lets you store and retrieve arbitrary data on a per-site-visitor basis. Data is stored on the server side and a session ID is stored in a cookie. The cookie contains only the session ID, not the actual data (unless using a cookie-based backend).

8. What are Django Signals? Signals allow certain senders to notify a set of receivers when some action has taken place. They are used to trigger actions when certain events occur (e.g., model save, delete).

Common built-in signals:

- `django.db.models.pre_init` & `post_init` – Sent before/after a model's `__init__()` method. - `pre_save` & `post_save` – Sent before/after a model's `save()` method. - `pre_delete` & `post_delete` – Sent before/after a model's `delete()` or queryset `delete()`. - `m2m_changed` – Sent when a `ManyToManyField` is changed. - `django.core.signals.request_started` & `request_finished` – Sent when an HTTP request starts or finishes.

9. Explain the caching strategies in Django. Caching stores previously computed results so they can be reused, improving performance.

Common caching strategies in Django:

- **Memcached** – Memory-based cache server; fastest and most efficient. - **FileSystem Caching** – Cache values stored as serialized files. - **Local-memory Caching** – Default strategy; per-process and thread-safe. - **Database Caching** – Cache data stored in the database; works well with a fast, indexed DB server.

10. Explain user authentication in Django. Django has a built-in authentication system that handles:

- Users - Groups - Permissions - Cookie-based sessions

It both authenticates (verifies identity) and authorizes (checks permissions) users. It includes a password hashing system, forms for login/registration, and a pluggable backend system.

11. How to configure static files?

1. Ensure `django.contrib.staticfiles` is in `INSTALLED_APPS`. 2. Define `STATIC_URL` in `settings.py`, for example:

```
STATIC_URL = '/static/'
```

3. In templates, use the `{% static %}` tag:

```
{% load static %}
<img src="{% static 'my_sample/abcxy.jpg' %}" alt="ABC image">
```

4. Store static files in a `static` directory inside your app, e.g. `my_sample/static/my_sample/abcxy.jpg`.

12. Explain Django response lifecycle.

1. Django loads `settings.py`, including middleware (`MIDDLEWARE`). 2. The request passes through middleware in order. 3. The URL router extracts the path and matches it to patterns in `urls.py`. 4. The matched view function is called with the `HttpRequest` object. 5. The view returns an `HttpResponse`. 6. The response passes back through middleware and is returned to the client.

13. What databases are supported by Django? Officially supported:

- PostgreSQL - MySQL - SQLite - Oracle

Via third-party backends, Django can also work with ODBC, Microsoft SQL Server, IBM DB2, SAP SQL Anywhere, Firebird, etc. (Note: Django does not officially support NoSQL databases.)

Advanced Django Interview Questions

1. Why is permanent redirection not always a good option? Permanent redirection (HTTP 301) is cached by browsers. If you later want to redirect the same URL to a different location, the browser may still use the old redirect, causing issues. It should only be used when you are sure the old URL will never be used again.

2. How to use file-based sessions? Set the `SESSION_ENGINE` setting to:

```
SESSION_ENGINE = "django.contrib.sessions.backends.file"
```

This tells Django to store session data in the filesystem.

3. What is a mixin? A mixin is a type of multiple inheritance where a class is used to provide additional methods/behavior to other classes. It's a way to reuse code from multiple classes. A downside is that it can become harder to understand which class provides which behavior if overused.

4. What is Django Field class? `Field` is an abstract class representing a column in a database table. It is a subclass of `RegisterLookupMixin`.

In Django:

- Fields define how database tables are created (`db_type()`). - They map Python types to database types using `get_prep_value()`. - They map database values back to Python objects using `from_db_value()`.

Fields are core to APIs like models and querysets.

5. Difference between `OneToOneField` and `ForeignKey` in Django?

- **ForeignKey** – Many-to-one relationship. Requires `on_delete` option and a target model. - **OneToOneField** – One-to-one relationship. Each object of the related model can be associated with only one instance of this model.

6. How can you combine multiple QuerySets in a view? You can concatenate QuerySets as lists, for example using `itertools.chain`:

```
from itertools import chain

result_list = list(chain(model1_list, model2_list, model3_list))
```

7. How to get a particular item in the Model?

```
ModelName.objects.get(id="term")
```

- Raises `DoesNotExist` if no result matches. - Raises `MultipleObjectsReturned` if more than one result matches.

8. How to obtain the SQL query from a queryset?

```
print(queryset.query)
```

9. What are the ways to customize the functionality of the Django admin interface?

- Customize the automatically generated add/change forms. - Include custom JavaScript via the `Media` class and its `js` attribute (list of script URLs). - Override admin templates. - Register custom admin views. - Customize `ModelAdmin` options (list display, filters, search, actions, etc.).

10. Difference between `select_related` and `prefetch_related`?

Both are used for optimizing queries involving related objects:

- **`select_related`** - Follows foreign key relationships. - Performs a SQL `JOIN` and selects related objects in a single query. - Best for one-to-one and many-to-one relations.

- **`prefetch_related`** - Performs separate queries and does the joining in Python. - Useful for many-to-many and reverse foreign key relationships.

Example:

```
from django.db import models

class Country(models.Model):
    country_name = models.CharField(max_length=5)

class State(models.Model):
    state_name = models.CharField(max_length=5)
    country = models.ForeignKey(Country, on_delete=models.CASCADE)

# Using select_related
states = State.objects.select_related('country').all()
for state in states:
    print(state.state_name)

# Using prefetch_related
country = Country.objects.prefetch_related('state_set').get(id=1)
for state in country.state_set.all():
    print(state.state_name)
```

11. Explain Q objects in Django ORM. `Q` objects are used to build complex queries with `OR` and other logical operators. Normal `filter()` arguments are ANDed; `Q` lets you OR them or negate them.

Example:

```
from django.db.models import Q

objects = Model.objects.get(
    Q(tag__startswith='Human'),
    Q(category='Eyes') | Q(category='Nose')
)
```

This generates:

```
SELECT * FROM Model
WHERE tag LIKE 'Human%'
  AND (category='Eyes' OR category='Nose');
```

12. What are Django exceptions? In addition to standard Python exceptions, Django defines its own, such as:

- `ObjectDoesNotExist` - `MultipleObjectsReturned` - `ValidationError` - `ImproperlyConfigured` - `FieldDoesNotExist` - etc.

These are raised by various parts of Django (models, forms, configuration) to signal framework-specific error conditions.